

Code Explanation

Testing Address Change Transformations

This test suite is designed to verify the correctness of the `apply\_address\_change\_transformations` function, which is responsible for transforming address change data. The test code is written using the Pytest framework and utilizes PySpark for data processing.

The test code creates a Spark session and sample data for testing the `apply\_address\_change\_transformations` function.

Overview of the Code

The provided code snippet is a part of a test suite written in Python using Pytest. It creates a Spark session and sample data for testing the `apply\_address\_change\_transformations` function. The sample data includes two records with different scenarios: one with complete data and another with missing ZIP code.

Step 1: Creating a Spark Session

The test code starts by creating a Spark session using the `SparkSession.builder` API. This Spark session is used throughout the test suite.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing"""
    return (SparkSession.builder
            .master("local[2]")
            .appName("E2E Address Change YUYU Tests")
            .getOrCreate())
```

Step 2: Defining the Sample Data Schema

The sample data schema is defined using the `StructType` and `StructField` classes from PySpark. The schema includes various fields such as request ID, transaction ID, policy ID, and address-related fields.

```
source_schema = StructType([
    StructField("T_TX_REQUEST_REQUEST_ID", StringType(), False),
    StructField("T_TX_BASIC_TRANS_ID", StringType(), False),
    StructField("T_TX_RELATION_TRANS_REL_ID", StringType(), False),
    StructField("T_TX_REQ_POL_REQUEST_POLICY_ID", StringType(), False),
    StructField("STG_POLICY_ID", StringType(), False),
    StructField("T_TX_REQUEST_ORIGIN_REQUEST_ID", StringType(), True),
    StructField("T_TX_BASIC_POLICY_ID", StringType(), True),
    StructField("T_TX_REQUEST_REQ_ACC_DATETIME", TimestampType(), True),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_ZIP", StringType(), True),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_ADD1", StringType(), True),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_AD1_KJ", StringType(), True),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_ADD2", StringType(), True),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_AD2_KJ", StringType(), True)
])
```

```

    StructField("T_TX_DTL_ADD_CH_N_TRANS_ADD3", StringType(), True),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_AD3_KJ", StringType(), True
),
    StructField("T_TX_DTL_ADD_CH_N_TRANS_PHN0", StringType(), True),
    StructField("PPAY_ZIP", StringType(), True),
    StructField("PPAY_ADR1_FW", StringType(), True),
    StructField("PPAY_ADR2_FW", StringType(), True),
    StructField("PPAY_ADR3_FW", StringType(), True),
    StructField("STG_TXDB_STATUS", StringType(), True),
    StructField("STG_PROCESS_DATE", TimestampType(), True),
    StructField("STG_PROCESS_USERID", StringType(), True)
1)

```

Step 3: Creating Sample Data

The sample data is created using the defined schema. The data includes two records with different scenarios.

```

source_data = [
    # Record 1: Complete data with ZIP
    (
        "REQ001", "TRANS001", "REL001", "REQPOL001", "POL001",
        "ORIG001", "12345678", datetime.datetime(2023, 5, 1, 10, 30,
0),
        "123-4567", "■■■■■■■", "■■■■■■■", "■■■1-1-1", "■■■1-1-1",
        "", "",
        "0312345678", "999-9999", "Fallback Addr1", "Fallback Addr2",
        "Fallback Addr3",
        "PENDING", None, None
    ),
    # Record 2: Missing ZIP, should use fallback
    (
        "REQ002", "TRANS002", "REL002", "REQPOL002", "POL002",
        "ORIG002", "87654321", datetime.datetime(2023, 5, 2, 11, 45,
0),
        None, None, "■■■■■■■", None, "■■■1-2-3", None, "■■■■4F",
        "0611112222", "888-8888", "■■■■■■■", "■■■1-2-3", "■■■■4F"
    ),
    "PENDING", None
)
1)

```